Correlación Canónica y Análisis Factorial

Ejemplo y actividad

In [2]:
```
# Bibliotecas necesarias
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.datasets import load_iris, load_breast_cancer, load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import FactorAnalysis, PCA
from sklearn.cross_decomposition import CCA
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, r2_score, roc_auc_score

import statsmodels.api as sm
from scipy import stats

%matplotlib inline
sns.set_style('whitegrid')
```

In [3]:
```
# ============================================================
# EJEMPLO 1: ANÁLISIS DE CORRELACIÓN CANÓNICA (CCA)
# ============================================================

# Dataset Iris: medimos dos conjuntos de variables
iris = load_iris()
df_iris = pd.DataFrame(iris.data, columns=iris.feature_names)
df_iris.columns = ['Long_Sepalo', 'Ancho_Sepalo', 'Long_Petalo', 'Ancho_P

# Conjunto X: sépalos. Conjunto Y: pétalos
X_cca = df_iris[['Long_Sepalo', 'Ancho_Sepalo']].values
Y_cca = df_iris[['Long_Petalo', 'Ancho_Petalo']].values

print("Conjunto X (sépalos):", X_cca.shape)
print("Conjunto Y (pétalos):", Y_cca.shape)
print("\nPrimeras filas:")
df_iris.head()
```

```
Conjunto X (sépalos): (150, 2)
Conjunto Y (pétalos): (150, 2)

Primeras filas:
```

Out[3]:

	Long_Sepalo	Ancho_Sepalo	Long_Petalo	Ancho_Petalo
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

In [4]:
```
# Estandarizar antes de CCA
scaler = StandardScaler()
X_cca_std = scaler.fit_transform(X_cca)
Y_cca_std = scaler.fit_transform(Y_cca)

# Ajustar CCA con 2 componentes
cca = CCA(n_components=2)
U, V = cca.fit_transform(X_cca_std, Y_cca_std)

# Correlaciones canónicas
rho = [np.corrcoef(U[:, i], V[:, i])[0, 1] for i in range(2)]
print("Correlaciones canónicas:")
for i, r in enumerate(rho):
    print(f"  ρ_{i+1} = {r:.4f}  (ρ² = {r**2:.4f})")
```

```
Correlaciones canónicas:
  ρ_1 = 0.9410  (ρ² = 0.8854)
  ρ_2 = 0.1239  (ρ² = 0.0154)
```

In [5]:
```
# Pesos canónicos (coeficientes)
pesos_X = pd.DataFrame(cca.x_weights_,
                       index=['Long_Sepalo', 'Ancho_Sepalo'],
                       columns=['U1', 'U2'])
pesos_Y = pd.DataFrame(cca.y_weights_,
                       index=['Long_Petalo', 'Ancho_Petalo'],
                       columns=['V1', 'V2'])

print("Pesos canónicos para X (sépalos):")
print(pesos_X.round(4))
print("\nPesos canónicos para Y (pétalos):")
print(pesos_Y.round(4))

# Cargas canónicas (correlación entre variable original y variable canóni
cargas_X = pd.DataFrame(
    [np.corrcoef(X_cca_std[:, j], U[:, i])[0, 1]
     for i in range(2) for j in range(2)],
    index=['U1-Long_Sepalo', 'U1-Ancho_Sepalo', 'U2-Long_Sepalo', 'U2-Anc
    columns=['Carga']
)
print("\nCargas canónicas para X:")
print(cargas_X.round(4))
```

```
Pesos canónicos para X (sépalos):
                  U1      U2
Long_Sepalo   0.9217  0.3880
Ancho_Sepalo -0.3880  0.9217

Pesos canónicos para Y (pétalos):
                  V1      V2
Long_Petalo   0.9430  0.3327
Ancho_Petalo -0.3327  0.9430

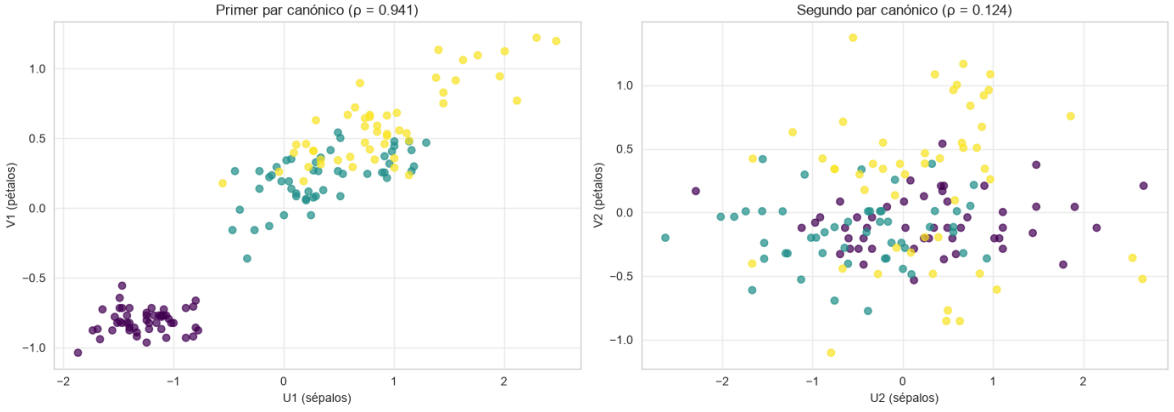
Cargas canónicas para X:
                   Carga
U1-Long_Sepalo   0.9290
U1-Ancho_Sepalo -0.4767
U2-Long_Sepalo   0.3701
U2-Ancho_Sepalo  0.8791
```

In [6]:
```python
# Scatter de las variables canónicas
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Par canónico 1
axes[0].scatter(U[:, 0], V[:, 0], c=iris.target, cmap='viridis', alpha=0.
axes[0].set_xlabel('U1 (sépalos)')
axes[0].set_ylabel('V1 (pétalos)')
axes[0].set_title(f'Primer par canónico (ρ = {rho[0]:.3f})')
axes[0].grid(True, alpha=0.3)

# Par canónico 2
axes[1].scatter(U[:, 1], V[:, 1], c=iris.target, cmap='viridis', alpha=0.
axes[1].set_xlabel('U2 (sépalos)')
axes[1].set_ylabel('V2 (pétalos)')
axes[1].set_title(f'Segundo par canónico (ρ = {rho[1]:.3f})')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In [7]:
```python
# ============================================================
# PARTE 2: ANÁLISIS FACTORIAL (AF)
# ============================================================

# Dataset Breast Cancer: 30 variables correlacionadas
data_bc = load_breast_cancer()
X_bc = pd.DataFrame(data_bc.data, columns=data_bc.feature_names)
y_bc = data_bc.target

print(f"Forma del dataset: {X_bc.shape}")
```

```
print(f"Variables: {X_bc.shape[1]}")
print(f"\nPrimeras variables:")
print(list(X_bc.columns[:10]))
```

```
Forma del dataset: (569, 30)
Variables: 30

Primeras variables:
['mean radius', 'mean texture', 'mean perimeter', 'mean area', 'mean smoot
hness', 'mean compactness', 'mean concavity', 'mean concave points', 'mean
symmetry', 'mean fractal dimension']
```

In [8]:
```
# Seleccionar 10 variables representativas (evitar multicolinealidad extr
vars_fa = ['mean radius', 'mean texture', 'mean perimeter',
           'mean area', 'mean smoothness', 'mean compactness',
           'mean concavity', 'mean concave points',
           'mean symmetry', 'mean fractal dimension']

X_fa = X_bc[vars_fa].values
X_fa_std = StandardScaler().fit_transform(X_fa)

print(f"Variables seleccionadas: {len(vars_fa)}")
print(f"Forma: {X_fa_std.shape}")
```

```
Variables seleccionadas: 10
Forma: (569, 10)
```

In [9]:
```
# Gráfico de sedimentación (scree plot)
pca_temp = PCA()
pca_temp.fit(X_fa_std)
eigenvalues = pca_temp.explained_variance_
varianza_acum = np.cumsum(pca_temp.explained_variance_ratio_)

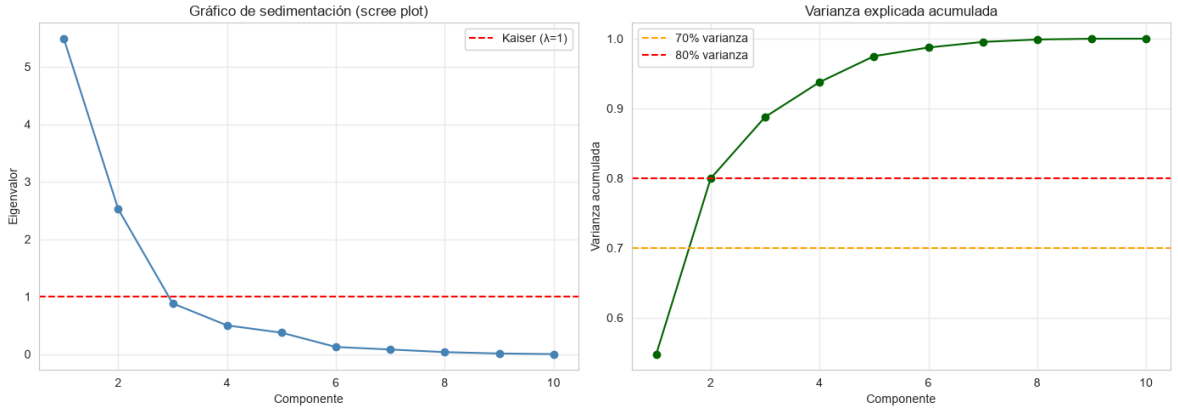
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Scree plot
axes[0].plot(range(1, len(eigenvalues) + 1), eigenvalues, 'o-', color='st
axes[0].axhline(y=1, color='r', linestyle='--', label='Kaiser (λ=1)')
axes[0].set_xlabel('Componente')
axes[0].set_ylabel('Eigenvalor')
axes[0].set_title('Gráfico de sedimentación (scree plot)')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Varianza acumulada
axes[1].plot(range(1, len(varianza_acum) + 1), varianza_acum, 'o-', color
axes[1].axhline(y=0.7, color='orange', linestyle='--', label='70% varianz
axes[1].axhline(y=0.8, color='red', linestyle='--', label='80% varianza')
axes[1].set_xlabel('Componente')
axes[1].set_ylabel('Varianza acumulada')
axes[1].set_title('Varianza explicada acumulada')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Regla de Kaiser
n_factores = sum(eigenvalues > 1)
print(f"\nFactores con eigenvalor > 1: {n_factores}")
print(f"Varianza acumulada con {n_factores} factores: {varianza_acum[n_fa
```




```
Factores con eigenvalor > 1: 2
Varianza acumulada con 2 factores: 0.800
```

In [10]:
```
# Extraer factores con FactorAnalysis (máxima verosimilitud)
fa = FactorAnalysis(n_components=n_factores, random_state=42)
scores_fa = fa.fit_transform(X_fa_std)

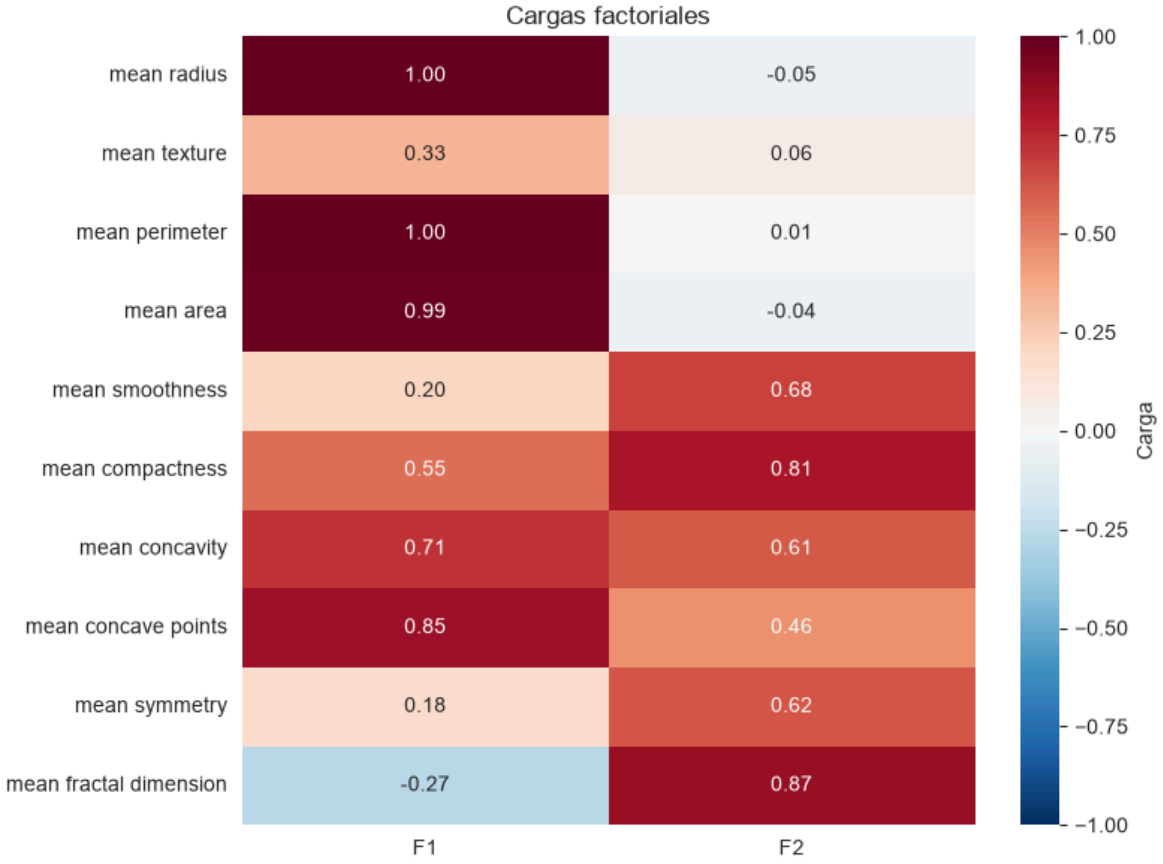
# Cargas factoriales
cargas_fa = pd.DataFrame(fa.components_.T,
                         index=vars_fa,
                         columns=[f'F{i+1}' for i in range(n_factores)])
```

```
print("Cargas factoriales:")
print(cargas_fa.round(3))
```

```
Cargas factoriales:
                           F1     F2
mean radius             0.998 -0.053
mean texture            0.329  0.060
mean perimeter          1.000  0.007
mean area               0.987 -0.044
mean smoothness         0.203  0.679
mean compactness        0.551  0.814
mean concavity          0.712  0.615
mean concave points     0.848  0.458
mean symmetry           0.179  0.619
mean fractal dimension -0.267  0.866
```

In [11]:
```
# Heatmap de cargas factoriales
plt.figure(figsize=(8, 6))
sns.heatmap(cargas_fa, annot=True, cmap='RdBu_r', center=0,
            vmin=-1, vmax=1, fmt='.2f', cbar_kws={'label': 'Carga'})
plt.title('Cargas factoriales')
plt.tight_layout()
plt.show()

# Comunalidades (proporción de varianza explicada por los factores)
comunalidades = (cargas_fa ** 2).sum(axis=1)
print("\nComunalidades (varianza explicada por factores):")
print(comunalidades.round(3))
```



```
Comunalidades (varianza explicada por factores):
mean radius               0.999
mean texture              0.112
mean perimeter            1.000
mean area                 0.976
mean smoothness           0.502
mean compactness          0.967
mean concavity            0.884
mean concave points       0.929
mean symmetry             0.415
mean fractal dimension    0.821
dtype: float64
```

In [12]:

```python
# ============================================================
# EJEMPLO 3: ANÁLISIS FACTORIAL APLICADO A REGRESIÓN
# ============================================================

# Usamos los scores factoriales como predictores
# Objetivo: clasificar si el tumor es benigno o maligno

# Modelo 1: regresión con variables originales (multicolinealidad)
X_train_orig, X_test_orig, y_train, y_test = train_test_split(
    X_fa_std, y_bc, test_size=0.3, random_state=42, stratify=y_bc
)

modelo_orig = LogisticRegression(max_iter=1000, random_state=42)
modelo_orig.fit(X_train_orig, y_train)
acc_orig = accuracy_score(y_test, modelo_orig.predict(X_test_orig))

# Modelo 2: regresión con scores factoriales
X_train_fa, X_test_fa, _, _ = train_test_split(
    scores_fa, y_bc, test_size=0.3, random_state=42, stratify=y_bc
)

modelo_fa = LogisticRegression(max_iter=1000, random_state=42)
modelo_fa.fit(X_train_fa, y_train)
acc_fa = accuracy_score(y_test, modelo_fa.predict(X_test_fa))

print("=== COMPARACIÓN DE MODELOS ===\n")
print(f"Modelo con variables originales ({X_fa.shape[1]} vars):")
print(f"  Accuracy: {acc_orig:.4f}")
print(f"\nModelo con factores ({n_factores} factores):")
print(f"  Accuracy: {acc_fa:.4f}")
print(f"\nReducción de dimensionalidad: {X_fa.shape[1]} → {n_factores}")
```

```
=== COMPARACIÓN DE MODELOS ===

Modelo con variables originales (10 vars):
  Accuracy: 0.9181

Modelo con factores (2 factores):
  Accuracy: 0.9181

Reducción de dimensionalidad: 10 → 2
```

In [13]:

```python
# ============================================================
# ACTIVIDAD: REPLICAR TODO CON EL DATASET DE VINOS
# ============================================================

print("ACTIVIDAD: Correlación Canónica y Análisis Factorial")
```

```
print("=" * 55)
print("\nDataset: Wine (load_wine)")
print("\nInstrucciones:")
print("\n1. CCA:")
print("   - Definir dos conjuntos de variables de vino.")
print("     Ejemplo: X = alcalinidad, magnesio, fenoles totales")
print("              Y = alcohol, ácido málico, ceniza")
print("   - Estandarizar ambos conjuntos")
print("   - Ajustar CCA con 2 componentes")
print("   - Reportar correlaciones canónicas")
print("   - Graficar pares canónicos")
print()
print("2. Análisis Factorial:")
print("   - Seleccionar 8-10 variables del dataset")
print("   - Estandarizar")
print("   - Scree plot para elegir número de factores")
print("   - Extraer factores y reportar cargas")
print("   - Interpretar los factores")
print()
print("3. FA + Regresión:")
print("   - Binificar la variable Clase (ej. 0 vs resto)")
print("   - Ajustar regresión logística con variables originales")
print("   - Ajustar regresión logística con scores factoriales")
print("   - Comparar accuracy de ambos modelos")
```

```
ACTIVIDAD: Correlación Canónica y Análisis Factorial
=======================================================

Dataset: Wine (load_wine)

Instrucciones:

1. CCA:
   - Definir dos conjuntos de variables de vino.
     Ejemplo: X = alcalinidad, magnesio, fenoles totales
              Y = alcohol, ácido málico, ceniza
   - Estandarizar ambos conjuntos
   - Ajustar CCA con 2 componentes
   - Reportar correlaciones canónicas
   - Graficar pares canónicos

2. Análisis Factorial:
   - Seleccionar 8-10 variables del dataset
   - Estandarizar
   - Scree plot para elegir número de factores
   - Extraer factores y reportar cargas
   - Interpretar los factores

3. FA + Regresión:
   - Binificar la variable Clase (ej. 0 vs resto)
   - Ajustar regresión logística con variables originales
   - Ajustar regresión logística con scores factoriales
   - Comparar accuracy de ambos modelos
```

In [26]:
```
# ============================================================
# PLANTILLA PARA LA ACTIVIDAD (COMPLETAR)
# ============================================================

# 1. CARGAR DATOS
wine = load_wine()
```

```
X_wine = pd.DataFrame(wine.data, columns=wine.feature_names)
y_wine = pd.Series(wine.target, name='Clase')
df_wine = pd.concat([X_wine, y_wine], axis=1)

print(df_wine.shape)
df_wine.head()
```

```
(178, 14)
```

Out[26]:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69

In [27]:

```
# 2. CCA - Definir conjuntos
X_cca_w = df_wine[['alcalinity_of_ash', 'magnesium', 'total_phenols']].va
Y_cca_w = df_wine[['alcohol', 'malic_acid', 'ash']].values

print("Conjunto X:", X_cca_w.shape)
print("Conjunto Y:", Y_cca_w.shape)
```

```
Conjunto X: (178, 3)
Conjunto Y: (178, 3)
```

In [28]:

```
# 3. CCA - Estandarizar y ajustar
X_cca_w_std = StandardScaler().fit_transform(X_cca_w)
Y_cca_w_std = StandardScaler().fit_transform(Y_cca_w)

cca_w = CCA(n_components=2)
U_w, V_w = cca_w.fit_transform(X_cca_w_std, Y_cca_w_std)

rho_w = [np.corrcoef(U_w[:, i], V_w[:, i])[0, 1] for i in range(2)]
print("Correlaciones canónicas:")
for i, r in enumerate(rho_w):
    print(f"  ρ_{i+1} = {r:.4f}  (ρ² = {r**2:.4f})")

# Pesos canónicos
pesos_X_w = pd.DataFrame(cca_w.x_weights_,
                         index=['alcalinity_of_ash', 'magnesium', 'total_
                         columns=['U1', 'U2'])
pesos_Y_w = pd.DataFrame(cca_w.y_weights_,
                         index=['alcohol', 'malic_acid', 'ash'],
                         columns=['V1', 'V2'])
print("\nPesos canónicos X:\n", pesos_X_w.round(4))
print("\nPesos canónicos Y:\n", pesos_Y_w.round(4))

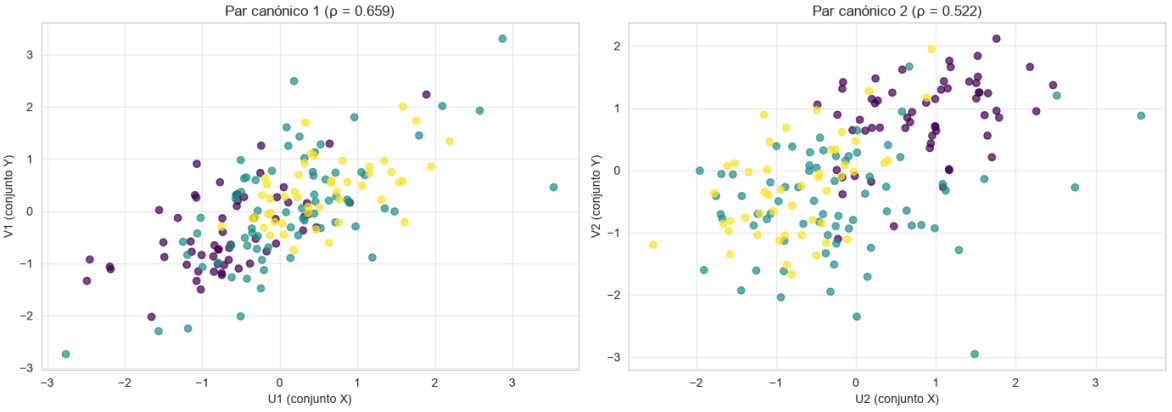
# Gráfica de los pares canónicos
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for i in range(2):
    axes[i].scatter(U_w[:, i], V_w[:, i], c=y_wine, cmap='viridis', alpha
    axes[i].set_xlabel(f'U{i+1} (conjunto X)')
    axes[i].set_ylabel(f'V{i+1} (conjunto Y)')
    axes[i].set_title(f'Par canónico {i+1} (ρ = {rho_w[i]:.3f})')
    axes[i].grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

```
Correlaciones canónicas:
  ρ_1 = 0.6592  (ρ² = 0.4346)
  ρ_2 = 0.5216  (ρ² = 0.2720)

Pesos canónicos X:
                        U1      U2
alcalinity_of_ash  0.9880 -0.0834
magnesium          0.1544  0.5278
total_phenols      0.0011  0.8453

Pesos canónicos Y:
                 V1      V2
alcohol    -0.5683  0.5025
malic_acid  0.3243 -0.5910
ash         0.7562  0.6310
```



Interpretación CCA

Correlaciones canónicas:

- $\rho_1 = 0.659, \rho^2=0.435$ Estos coeficientes muestran que el primer par de variables canónicas comparten cerca del 43% de su varianza. Lo que significa que es una relación moderadamente fuerte entre las variables del conjunto X (`alcalinidad` , `magnesio` , `fenoles`) y las del conjunto Y (`alcohol` , `ácido` , `ceniza`)
- $\rho_2 = 0.522, \rho^2=0.272$ mientras que estos coeficientes también muestran una relación moderada. No es despreciable, por lo que se concluye que hay dos dimensiones independientes de asociación entre ambos conjuntos.

Pesos canónicos:

$U1$ está dominada casi por completo por `alcalinity_of_ash` ya que se presenta un coeficiente de 0.988, por lo que esta misma representa casi por completo la alcalinidad de la ceniza mientras que en $V1$ se contrasta `ash` con $+0.756$, `alcohol` con -0.568 y `malic_acid` con 0.528.

Por otro lado $U2$ se forma sobre todo con `total_phenols` 0.845 y `magnesium` con 0.528; también se muestra que $V2$ es una mezcla entre

`ash` con $+0.631$, `alcohol` con $+0.503$ contra `malic_acid` con -0.591

In [29]:
```
# 4. FA - Seleccionar variables
vars_fa_w = ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash',
             'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_ph

X_fa_w = df_wine[vars_fa_w].values
X_fa_w_std = StandardScaler().fit_transform(X_fa_w)

print(f"Variables seleccionadas: {len(vars_fa_w)}")
print(f"Forma: {X_fa_w_std.shape}")
```

```
Variables seleccionadas: 8
Forma: (178, 8)
```

In [31]:
```
# 5. FA - Scree plot
pca_w = PCA().fit(X_fa_w_std)
varianza_acum_w = np.cumsum(pca_w.explained_variance_ratio_)

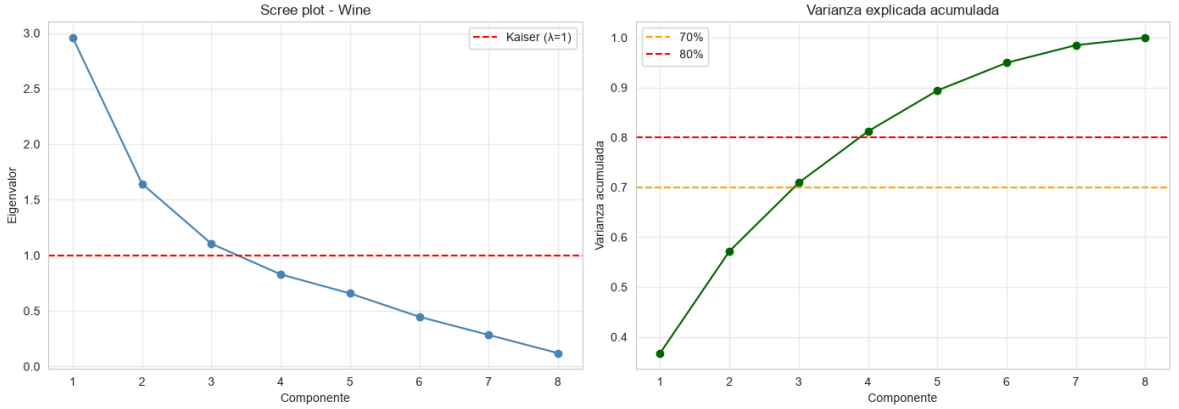
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(range(1, len(pca_w.explained_variance_) + 1),
             pca_w.explained_variance_, 'o-', color='steelblue')
axes[0].axhline(y=1, color='r', linestyle='--', label='Kaiser (λ=1)')
axes[0].set_xlabel('Componente')
axes[0].set_ylabel('Eigenvalor')
axes[0].set_title('Scree plot - Wine')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(range(1, len(varianza_acum_w) + 1), varianza_acum_w,
             'o-', color='darkgreen')
axes[1].axhline(y=0.7, color='orange', linestyle='--', label='70%')
axes[1].axhline(y=0.8, color='red', linestyle='--', label='80%')
axes[1].set_xlabel('Componente')
axes[1].set_ylabel('Varianza acumulada')
axes[1].set_title('Varianza explicada acumulada')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

n_fact_w = int(sum(pca_w.explained_variance_ > 1))
print(f"Factores con eigenvalor > 1: {n_fact_w}")
print(f"Varianza acumulada con {n_fact_w} factores: {varianza_acum_w[n_fa
```



```
Factores con eigenvalor > 1: 3
Varianza acumulada con 3 factores: 0.709
```

Scree Plot

Los eigenvalores de los tres primeros componentes son aproximadamente 2.95, 1.64 y 1.1 por lo que son los componentes más representativos ya que del cuarto componente en adelante los eigenvalores son menores a 1, también gracias a la gráfica se puede notar claramente que el codo después del tercer componente y se suaviza después del tercero; por lo que se tiene un compromiso razonable entre simplicidad y varianza explicada.

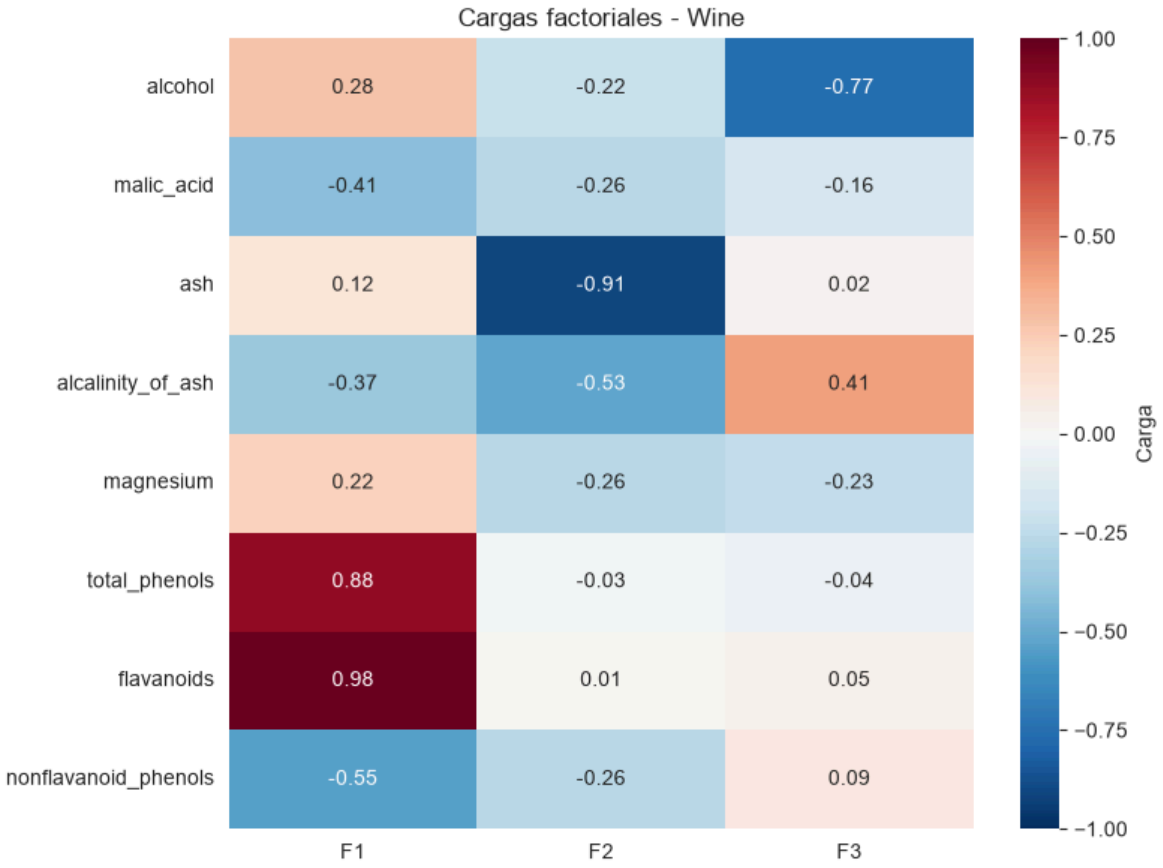
In [32]:
```python
# 6. FA - Extraer factores
fa_w = FactorAnalysis(n_components=n_fact_w, random_state=42)
scores_w = fa_w.fit_transform(X_fa_w_std)

cargas_w = pd.DataFrame(fa_w.components_.T,
                        index=vars_fa_w,
                        columns=[f'F{i+1}' for i in range(n_fact_w)])
print("Cargas factoriales:")
print(cargas_w.round(3))

# Heatmap de cargas
plt.figure(figsize=(8, 6))
sns.heatmap(cargas_w, annot=True, cmap='RdBu_r', center=0,
            vmin=-1, vmax=1, fmt='.2f', cbar_kws={'label': 'Carga'})
plt.title('Cargas factoriales - Wine')
plt.tight_layout()
plt.show()

# Comunalidades
comunalidades_w = (cargas_w ** 2).sum(axis=1)
print("\nComunalidades:")
print(comunalidades_w.round(3))
```

```
Cargas factoriales:
                           F1     F2     F3
alcohol                 0.281 -0.216 -0.765
malic_acid             -0.410 -0.258 -0.160
ash                     0.125 -0.911  0.025
alcalinity_of_ash      -0.373 -0.527  0.414
magnesium               0.223 -0.259 -0.230
total_phenols           0.882 -0.028 -0.040
flavanoids              0.981  0.010  0.048
nonflavanoid_phenols   -0.547 -0.265  0.088
```




```
Comunalidades:
alcohol                 0.711
malic_acid              0.260
ash                     0.845
alcalinity_of_ash       0.588
magnesium               0.169
total_phenols           0.781
flavanoids              0.964
nonflavanoid_phenols    0.377
dtype: float64
```

Cargas factoriales y comunalidades

Las cargas que tengan un valor absoluto mayor a 0.5 se pueden considerar como relevantes, entre cada uno de los valores el más representativo se muestra en el siguiente listado:

- $F1$ - `flavanoids` con un coeficiente de 0.981 y `total_phenols` con 0.882
- $F2$ - `ash` con un coeficiente de -0.911 y `alcalinity_of_ash` con -0.527
- $F3$ - `alcohol` con un coeficiente de -0.765 y `alcalinity_of_ash` con 0.414

Las comunalidades que se encuentran bien representadas son `flavanoids` (0.964), `ash` (0.865), `total_phenols` (0.781) y `alcohol` (0.711) mientras que la variable peormente representada es `magnesium` (0.169)

In [33]:
```
# 7. FA + Regresión
df_wine['Clase_bin'] = (df_wine['Clase'] == 0).astype(int)
```

```python
# Modelo con variables originales
X_tr_w, X_te_w, y_tr_w, y_te_w = train_test_split(
    X_fa_w_std, df_wine['Clase_bin'], test_size=0.3,
    random_state=42, stratify=df_wine['Clase_bin']
)
m_orig_w = LogisticRegression(max_iter=1000, random_state=42).fit(X_tr_w,
acc_orig_w = accuracy_score(y_te_w, m_orig_w.predict(X_te_w))

# Modelo con scores factoriales (mismo split)
X_tr_wf, X_te_wf, _, _ = train_test_split(
    scores_w, df_wine['Clase_bin'], test_size=0.3,
    random_state=42, stratify=df_wine['Clase_bin']
)
m_fa_w = LogisticRegression(max_iter=1000, random_state=42).fit(X_tr_wf,
acc_fa_w = accuracy_score(y_te_w, m_fa_w.predict(X_te_wf))

print("=== COMPARACIÓN DE MODELOS ===")
print(f"Variables originales ({len(vars_fa_w)} vars): accuracy = {acc_ori
print(f"Factores ({n_fact_w} factores):  accuracy = {acc_fa_w:.4f}")
print(f"Reducción de dimensionalidad: {len(vars_fa_w)} → {n_fact_w}")
```

```
=== COMPARACIÓN DE MODELOS ===
Variables originales (8 vars): accuracy = 0.9444
Factores (3 factores):  accuracy = 0.9074
Reducción de dimensionalidad: 8 → 3
```

En conclusión se puede decir que ambas técnicas encontraron estructura en los datos del vino:

- La CCA mostró asociaciones moderadas entre dos grupos de propiedades químicas
- El AF resumió ocho variables en tres factores interpretables (`fenoles` , `minerales` , `alcohol`) con 70.9% de varianza explicada.